LLD Interview Structure & 12-Problem Revision Guide

A C++-focused revision handbook for interview practice

Goal: Learn a repeatable interview process, recognize the design problem before choosing a pattern, and explain why each design decision exists.

Core rule

Do not start with: Which design pattern does this problem use?
Start with: What are the requirements, entities, relationships, responsibilities, and changing behaviors?

Part I - Structure to Follow in an LLD Interview

Use the same sequence for almost every LLD problem:

```
Problem
  |
  v
1. Clarify Requirements
  |
2. Identify Core Entities
  |
3. Identify Relationships
  |
4. Identify Important Behaviors
  |
5. Find What Varies
  |
6. Choose Patterns if Needed
  |
7. Sketch UML / Class Design
  |
8. Code the Core Flow
  |
9. Walk Through an Example
  |
10. Discuss Extensions and C++ Concerns
```

1. Clarify requirements

Confirm scope before coding. Ask what operations are required, what can be ignored, and whether concurrency, persistence, failures, capacity, or multiple users matter. Example for ATM: card insertion, PIN authentication, withdrawal and card ejection are in scope; deposits and real bank communication may be excluded.

2. Identify core entities

Extract the important nouns. ATM might have ATM, Card, Account, Transaction and CashDispenser. Parking Lot might have Vehicle, ParkingSpot, Ticket and ParkingLot. Do not force patterns yet.

3. Identify relationships

Ask IS-A, HAS-A, OWNS, REFERENCES, and ONE-TO-MANY. In C++, also think about lifetime: who owns an object and who merely stores a non-owning pointer/reference?

4. Identify behaviors

List the important operations: insertCard(), enterPin(), withdraw(), park(), unpark(), calculateFee(), submit(), execute(), etc. This exposes where responsibilities belong.

5. Find what varies

Ask: What behavior is likely to change independently? Different pricing algorithms suggest Strategy. Lifecycle-dependent behavior suggests State. Multiple event listeners suggest Observer.

6. Choose patterns only when justified

Explain the design reason. Example: 'ATM behavior depends heavily on current state, so I will encapsulate state-specific behavior instead of spreading state checks across ATM methods.'

7. Sketch the class design

Draw a small UML/class relationship diagram before writing code. Show interfaces, concrete implementations, HAS-A relationships, and important collections.

8. Code incrementally

A useful C++ order is: abstractions/interfaces -> core entities -> concrete implementations -> coordinator/context -> main/demo. Avoid writing the entire solution in one shot.

9. Walk through one scenario

Trace one complete request through the design. This validates transitions, polymorphism, object relationships, and edge cases.

10. Discuss extensions and C++ concerns

After the basic solution works, discuss ownership, smart pointers, dangling references, thread safety, mutex/condition_variable, failure handling, bounded queues, and scaling where relevant.

Pattern recognition cheat sheet

Requirement signal	Likely tool/pattern
Multiple interchangeable algorithms/behaviors	Strategy
Behavior depends on current lifecycle/mode	State
Multiple components react to an event	Observer
Create different concrete objects	Factory
Represent an operation/request as an object	Command
Dynamically stack optional features	Decorator
Individual objects + recursive groups/tree	Composite
Request flows through handlers	Chain of Responsibility
Hide a complex subsystem behind a simple API	Facade

Part II - Revision of the 12 Level-2 Problems

For every problem, revise in the same order: requirements -> entities -> relationships/flow -> what varies -> pattern -> C++ concern.

1. Notification System

Primary pattern(s): Strategy

Requirements

Support Email, SMS and Push notifications. The caller can choose or switch the delivery mechanism, and adding a new channel should not require changing the notification service.

Core entities

NotificationService, INotificationStrategy, EmailNotification, SMSNotification, PushNotification.

Why this pattern?

The overall operation is the same - send a notification - but the algorithm/behavior used to send it varies. That is the core Strategy signal.

Core flow / relationship

NotificationService HAS-A INotificationStrategy. makeNotification() delegates to strategy->send(). setStrategy() can switch behavior at runtime.

What to say in the interview

Say: 'The delivery mechanism is interchangeable, so I will depend on an abstraction and inject a notification strategy rather than use a growing if/else chain.'

C++ / engineering concern

If the service stores a raw strategy pointer, clarify that it is non-owning and the strategy must outlive the service. If the service owns the strategy, unique_ptr is a better fit.

2. Vending Machine

Primary pattern(s): State

Requirements

Select a product, insert money, dispense it, and return to idle. Operations are valid or invalid depending on the machine's current condition.

Core entities

VendingMachine, State, IdleState, HasSelectionState, DispensingState.

Why this pattern?

The same operation behaves differently depending on the current state. State avoids scattering state checks across every method.

Core flow / relationship

Idle -> selectProduct -> HasSelection -> insertMoney -> Dispensing -> dispense -> Idle.

What to say in the interview

Say: 'The valid behavior is driven by the machine lifecycle, so I will encapsulate state-specific behavior and transitions in State objects.'

C++ / engineering concern

A small enum/switch can be sufficient for very simple machines. Use full State when state-specific behavior and transitions are expected to grow.

3. Parking Lot

Primary pattern(s): Domain modeling + Strategy

Requirements

Park Bike/Car/Truck in compatible spots, issue a ticket, release the spot on exit, and calculate a fee using a replaceable pricing policy.

Core entities

Vehicle, ParkingSpot, ParkingTicket, ParkingLot, PricingStrategy.

Why this pattern?

The system is primarily a modeling problem. Pricing is the clearly interchangeable behavior, so Strategy fits that part.

Core flow / relationship

parkVehicle() finds a compatible free spot -> marks it occupied -> returns a ticket. exitVehicle() calculates fee through PricingStrategy -> frees spot.

What to say in the interview

Do not say 'Parking Lot uses Strategy' first. Model entities and relationships first, then identify pricing as a varying policy.

C++ / engineering concern

Be careful returning ParkingSpot* into vector; vector reallocation can invalidate pointers. Stable ownership/container choices matter in production.

4. Logger System

Primary pattern(s): Chain of Responsibility

Requirements

Support INFO, DEBUG and ERROR handlers. A log request should move through a configured chain until the appropriate handler processes it.

Core entities

Logger base handler, InfoLogger, DebugLogger, ErrorLogger.

Why this pattern?

The request moves through a sequence of handlers. Each handler either handles it or forwards it.

Core flow / relationship

InfoLogger -> DebugLogger -> ErrorLogger. Each handler stores a next pointer.

What to say in the interview

Differentiate from Strategy: Strategy chooses one behavior; Chain sends a request through handlers.

C++ / engineering concern

If handlers are stack objects, next pointers are non-owning. Do not delete next. A factory can later hide construction of the chain.

5. Elevator System

Primary pattern(s): Strategy + state concepts

Requirements

Manage elevators across floors, accept external requests, move an elevator, and allow the elevator-selection algorithm to change.

Core entities

Elevator, ElevatorController, ElevatorSelectionStrategy, NearestElevatorStrategy, Direction.

Why this pattern?

Which elevator should serve a request is an interchangeable algorithm -> Strategy. UP/DOWN/IDLE are state concepts, but a simple enum may be enough until behavior becomes complex.

Core flow / relationship

Controller asks strategy to select an elevator -> selected elevator moves to requested floor -> later moves to passenger destination.

What to say in the interview

Explain that you would introduce full State only when door states, maintenance, emergency and transition rules make an enum/switch unwieldy.

C++ / engineering concern

Returning Elevator* into vector can become unsafe if later additions reallocate the vector.

6. Coffee Ordering System

Primary pattern(s): Decorator (+ Factory concept)

Requirements

Start with Espresso or Cappuccino and dynamically add Milk, Sugar, Vanilla, etc. Arbitrary combinations should not require a subclass for every combination.

Core entities

Coffee interface, Espresso, Cappuccino, CoffeeDecorator, Milk, Sugar, Vanilla.

Why this pattern?

Optional features need to be stacked dynamically around the same abstraction. Decorator avoids combinatorial subclass explosion.

Core flow / relationship

Vanilla -> Sugar -> Milk -> Espresso. getCost()/getDescription() delegate inward and add each decorator's contribution.

What to say in the interview

Memory rule: Strategy switches behavior; Decorator stacks behavior.

C++ / engineering concern

A decorator both IS-A Coffee and HAS-A Coffee. With raw pointers clarify ownership. Production code often models the ownership chain with unique_ptr.

7. File System

Primary pattern(s): Composite

Requirements

Files and directories share operations such as getName(), getSize() and display(). Directories can recursively contain files and other directories.

Core entities

FileSystemItem, File (leaf), Directory (composite).

Why this pattern?

Clients should treat an individual object and a group uniformly, and the structure is recursive/tree-shaped.

Core flow / relationship

Directory stores FileSystemItem children. getSize() recursively calls child->getSize(); display() recursively traverses children.

What to say in the interview

Recognition: 'A container can contain objects of the same general abstraction, including more containers' -> Composite.

C++ / engineering concern

vector is non-owning in the simple version. A natural production model is vector> when the directory owns children.

8. ATM System

Primary pattern(s): State + Chain of Responsibility

Requirements

Insert card, authenticate PIN, withdraw, eject card. Operations depend on ATM state. Cash is dispensed through 500/200/100 denomination handlers.

Core entities

ATM, ATMState, NoCardState, CardInsertedState, AuthenticatedState, CashDispenser handlers.

Why this pattern?

ATM lifecycle behavior -> State. Cash amount passing through denomination handlers -> Chain of Responsibility.

Core flow / relationship

NoCard -> CardInserted -> Authenticated. Withdrawal 1800: 500 handler dispenses 1500 -> 200 dispenses 200 -> 100 dispenses 100.

What to say in the interview

Treat them as two separate design problems. Do not say 'ATM uses State and Chain' without explaining which requirement each solves.

C++ / engineering concern

The simplified dispenser assumes unlimited notes and greedy selection. Real inventory can make greedy incorrect; the design pattern does not guarantee algorithmic correctness.

9. Library Management System

Primary pattern(s): Domain modeling + Strategy

Requirements

Books have metadata, multiple physical copies can exist, members borrow/return copies, active loans are tracked, and books can be searched by different criteria.

Core entities

Book, BookCopy, Member, Loan, Library/Catalog, SearchStrategy.

Why this pattern?

The key LLD insight is separating a publication from a physical copy and modeling Loan as the relationship between Member and BookCopy. Search behavior can use Strategy.

Core flow / relationship

Member requests ISBN -> library checks loan limit -> finds available BookCopy -> marks it unavailable -> creates Loan. Return reverses the relationship.

What to say in the interview

If a relationship has meaningful data such as due date, return date, renewal count or fine, model the relationship itself as an object.

C++ / engineering concern

Concurrent borrowers can race for the last copy. Availability check + marking unavailable + loan creation must eventually become one synchronized/transactional operation.

10. Traffic Signal Controller

Primary pattern(s): State + Observer

Requirements

Traffic light transitions RED -> GREEN -> YELLOW -> RED. Pedestrian display and monitoring components should react to state changes without tight coupling.

Core entities

TrafficSignal, SignalState, RedState, GreenState, YellowState, Observer, PedestrianDisplay, MonitoringSystem.

Why this pattern?

Lifecycle transitions -> State. Multiple independent listeners reacting to changes -> Observer.

Core flow / relationship

currentState->next() triggers transition. setState() notifies all observers with the new signal state.

What to say in the interview

For only three trivial states, an enum/switch may be simpler. State becomes more valuable with emergency, maintenance, pedestrian and failure modes.

C++ / engineering concern

Observer pointers are typically non-owning in the simple version, so subscribers must outlive their subscriptions.

11. Task Scheduler / Job Queue

Primary pattern(s): Command + Queue / concurrency concepts

Requirements

Different jobs should be submitted, queued and executed later. Scheduler should not know concrete task logic. Future version may have worker threads.

Core entities

Task/Command, ImageProcessingTask, EmailTask, DatabaseBackupTask, TaskScheduler.

Why this pattern?

An operation plus its data is represented as an object -> Command. Commands are naturally queueable, delayable, retryable and executable asynchronously.

Core flow / relationship

Producer submitTask() -> FIFO queue -> worker takes front -> pop -> execute().

What to say in the interview

If threads are added: protect queue with mutex, let workers sleep using condition_variable, and execute the task after releasing the queue lock.

C++ / engineering concern

queue creates lifetime risk for asynchronous work. A scheduler that owns pending jobs can use queue>. Bounded queues require an explicit backpressure/eviction policy.

12. Camera / Device Management System

Primary pattern(s): State + Command + Observer + Facade

Requirements

Camera has sensor, inference, network and storage subsystems; lifecycle states; commands from a hub; event listeners; and a simple high-level startup/shutdown interface.

Core entities

CameraDevice, CameraState states, Command hierarchy, Hub, Observer hierarchy, subsystem classes.

Why this pattern?

Lifecycle -> State. Hub operations as objects -> Command. Telemetry/logger/health reactions -> Observer. Hide subsystem startup complexity -> Facade.

Core flow / relationship

device.start() initializes subsystems and reaches READY. Hub executes StartStreamCommand -> device.startStreaming() -> State transitions READY to STREAMING -> observers receive event.

What to say in the interview

Start from responsibilities, not 'I will use four patterns.' Explain each pattern only where a specific requirement justifies it.

C++ / engineering concern

Commands and observers storing CameraDevice*/Observer* require lifetime guarantees. Real versions add queues, threads, bounded buffers, shutdown flags, condition variables, failure rollback and resource ownership.

Part III - Final Revision Checklist

Before moving to Level 3, take each of the 12 problems and try to answer these questions without looking at your saved solution:

- Requirements: Can I state the scope and important assumptions in under one minute?
- Entities: Can I identify the main classes without immediately thinking about patterns?
- Relationships: Can I explain IS-A, HAS-A, ownership and one-to-many relationships?
- Behavior: Can I list the important operations?
- Variation: Can I identify which behavior changes independently?
- Pattern: Can I explain why the pattern fits instead of merely naming it?
- UML: Can I sketch the core class relationships in 2-5 minutes?
- Code: Can I implement the core flow in roughly 20-30 minutes?
- Walkthrough: Can I trace one end-to-end scenario?
- C++: Can I explain ownership/lifetime and identify likely concurrency problems?

The interview mindset

```
Do NOT memorize:
    Parking Lot -> Strategy

Instead reason:
    Requirements
        -> Entities
        -> Relationships
        -> Responsibilities
        -> What varies?
        -> Appropriate abstraction/pattern
        -> Code
        -> Validate
        -> Discuss extensions
```

Once you can derive the designs this way, you are ready for Level 3, where requirements change after the initial design and you must extend the system without breaking the architecture.